

IRE Assignment Design Note

Iman Kalyan Chakraborty - 2026801003

August 26, 2026

<https://github.com/astrion-coder/cs4406m26-assignment1c1/>

1 Introduction

This assignment implements a complete lexical and semantic retrieval pipeline for news recommendation over two real-world datasets, EB-NeRD (demo, small, and large variants) and MIND (small and large variants). The two datasets differ in language, storage schema, and scale. The small and demo variants were used for experimentation; the large variants for the real Codabench submissions.

The pipeline has four stages: **unified schema conversion**, since the two datasets' raw formats differ and must share one format before use; **lexical candidate generation** via a from-scratch BM25 implementation, faster than the `rank_bm25` package; **semantic candidate generation** via `paraphrase-xlm-r-multilingual-v1` embeddings, multilingual since EB-NeRD is Danish and MIND is English; and **evaluation** against every required metric: AUC, MRR, nDCG@k, diversity, novelty, and coverage.

The rest of this report covers each stage's design decisions in Section 2, then the resulting experimental findings in Section 3, including a real large-scale engineering incident hit while running Q4 (Section 3.4).

2 Design

2.1 Unified Schema

EB-NeRD (Danish, Parquet, provider-supplied embeddings, no entities) and MIND (English, TSV, no embeddings of its own, knowledge-graph entities) share almost nothing at the raw-file level. An early choice was to convert both into one common schema and write every downstream stage (Q2 through Q5) against that schema only, not five dataset-specific code paths.

Three tables cover both. `articles` holds `article_id`, `title`, `abstract`, `body`, `category`, `published_time`, `dataset`. `behaviors` holds one row per impression: `impression_id`, `user_id`, `impression_time`, `article_ids_inview`, `article_ids_clicked`, `session_id`, `split`, `dataset`. `history` holds each user's pre-window click sequence: `article_id_sequence`, `timestamp_sequence`, `read_time_sequence`, `scroll_percentage_sequence`. A column only one raw format actually has is left null or absent rather than fabricated: MIND parses no `body` text, and the real, blind `ebnerd_testset` has no `article_ids_clicked`. A placeholder was considered and rejected, since it would hide exactly the cases where the absence itself is the load-bearing fact.

The datasets are never merged despite sharing a schema: each builds into its own output directory, and every ID is namespaced (`ebnerd_123`, `mind_N55528`), so a bug can never silently join rows across them. This also enforces Q9's anti-gaming requirement at the schema level, not as an afterthought: EB-NeRD's raw behaviors carry look-ahead fields, `next_read_time` and `next_scroll_percentage`, that would leak future engagement if used as features. The unified schema simply never carries them, and a test, `test_no_leakage_columns`, asserts this against the real on-disk schema every run.

2.2 Q1: Reproducible Data Pipeline

The split is never random. Each dataset's provider train split is followed by its provider dev/validation split, in that time order. The provider's dev/validation split becomes the held-out **test** set; **val** is carved from the last day of the provider **train** split, by time. A random split was rejected: it would leak future articles and trends into training.

Two leakage checks run at build time: EB-NeRD’s history timestamps must precede every impression for that user, and MIND’s per-row history string must be identical across all of a user’s rows, confirming a fixed pre-window snapshot. The pipeline rebuilds from raw files with one command, with a test cell after every step, so a broken build fails loudly instead of silently writing wrong data.

2.3 Q2: Lexical Candidate Generation (BM25)

BM25 is implemented from scratch in `bm25.py` instead of `rank_bm25`, purely for speed. `rank_bm25`’s plain per-term dictionary lookup measured near 3 seconds per query on MIND’s 65,238-article corpus, over 40 hours at the scale needed here. A real postings-list index, scored with a vectorized numpy operation and `np.argmaxpartition` for top-K, measured about 5ms per query instead, roughly a thousand times faster.

A user’s query is their most recent 20 clicked titles, concatenated. Since this depends only on a fixed history, the top-200 scores are computed once per user and cached; recall@50/100 are read off as prefixes of that list instead of re-scoring. Recall@K is a fraction, $|\text{clicked} \cap \text{top-K}|/|\text{clicked}|$, not a plain hit or miss, since one impression can have more than one clicked article.

2.4 Q3: Semantic Candidate Generation (Embeddings)

One embedding model, `paraphrase-xlm-r-multilingual-v1`, is used for both datasets (not EB-NeRD’s own provided embeddings), so Danish and English text share one vector space. MIND has no article embeddings of its own, only unrelated knowledge-graph entity vectors.

Embeddings are computed once on a Kaggle GPU notebook and saved as parquet, so the local environment never installs `sentence-transformers/torch`. Retrieval is a brute-force batched matrix multiply plus `argpartition`, not FAISS: at this corpus size (up to about 65,000 articles), the full MIND val/test population retrieves in well under two minutes, batched in groups of a few thousand users since the full similarity matrix would otherwise take roughly 17GB at once.

A user’s representation is the mean of their last 20 clicked embeddings, the same window Q2 uses, so Section 3.2’s comparison only varies the method.

2.5 Q4: Offline Evaluation Harness

AUC, MRR, and nDCG need a full ranking over candidates with known labels. The only such set per impression is `article_ids_inview` (what the user was shown), not the whole catalog. This differs from Q2/Q3’s recall@K , which asks whether a method can find the clicked article at all; here the question is whether it ranks the clicked article near the top among what was shown. Q4 cannot reuse Q2/Q3’s saved top-200 lists for this, since MIND’s recall@200 is only a few percent and the clicked article is usually not in it. Instead a re-scoring adapter, `score_inview`, scores each method for exactly one impression’s inview set, on the fly.

Beyond-accuracy: intra-list diversity is one minus the average same-category-pair share in the top-10 list; novelty is the negative log of an item’s train-click share (add-one smoothed); coverage is the catalog fraction appearing in the saved top-200 lists (computed once per dataset, not per split). Cold-start vs. warm is the required slice; head vs. tail (top 20% by train clicks vs. the rest) is a second. Every metric carries a 95% bootstrap CI over 1,000 resamples.

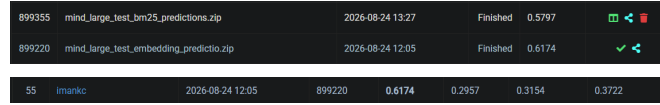
2.6 Q5: Codabench Submission

The submission format, one line per impression as `{impression_id} [rank_1,...,rank_n]` in the original row order, was confirmed directly from MIND’s Submission Guidelines page and matches both the EB-NeRD starter repository and the reference MIND example code. Prediction files were generated for all three dataset tracks with `generate_predictions.py`, reusing Q4’s `score_inview` adapters.

The Codabench competitions grade against a separate, hidden test population (`MINDlarge_test`, `ebnerd_testset.zip`), not the local provider dev/validation `test` split. MIND’s 26,228 test-only articles (of 120,961) are re-encoded on a fresh Kaggle GPU pass; the rest reuse existing embeddings. EB-NeRD needs no new embeddings, since its test catalog was verified byte-for-byte identical to `ebnerd_large`’s. Both wrote valid, round-trip-tested prediction zips (2,370,727 and 13,336,710 lines, matching each population’s impression count exactly), submitting `embedding` per its Q4-measured edge (Section 3.2).

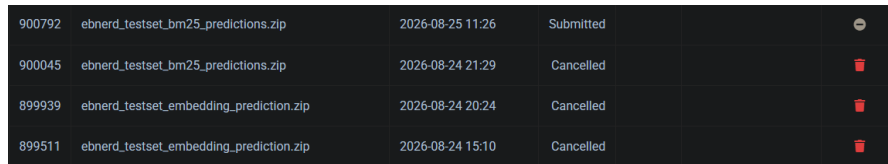
Leaderboard Screenshots

MIND: both methods scored on the real `MINDlarge_test` leaderboard. `embedding` reaches AUC 0.6174 (rank 55); `bm25` reaches AUC 0.5797.



899355	mind_large_test_bm25_predictions.zip	2026-08-24 13:27	Finished	0.5797		
899220	mind_large_test_embedding_predictio.zip	2026-08-24 12:05	Finished	0.6174		
55	imankc	2026-08-24 12:05	899220	0.6174	0.2957	0.3154 0.3722

EB-NeRD: Codabench’s own evaluation queue was still processing the submission at writing time (server-side scoring time, not under this pipeline’s control). The screenshot shows the submission accepted and queued, not yet a scored row; it will be updated once Codabench returns a score. The cancelled ones were cancelled after ~ 4 -5 hours of processing.



900792	ebnerd_testset_bm25_predictions.zip	2026-08-25 11:26	Submitted	
900045	ebnerd_testset_bm25_predictions.zip	2026-08-24 21:29	Cancelled	
899939	ebnerd_testset_embedding_prediction.zip	2026-08-24 20:24	Cancelled	
899511	ebnerd_testset_embedding_prediction.zip	2026-08-24 15:10	Cancelled	

2.7 Code Optimizations

Several real performance and memory problems were found and fixed while running the pipeline at full scale, in the order encountered.

- `rank_bm25`’s **per-term lookup was the Q2 bottleneck**: near 3 seconds per query, over 40 hours at the scale needed. A postings-list index with vectorized numpy and `argpartition` top-K cut this to about 5ms per query, roughly $1000\times$ faster.
- **Pandas hit a memory and throughput ceiling at `ebnerd_large` / `mind_large` scale**. The full pipeline was rewritten to polars (lazy scans, columnar operations), which made large-dataset runs tractable at all.
- **Writing Q2/Q3’s dense top-K list column in one shot ran out of memory, and `gc.collect()` cost minutes per call against large notebook state**. A chunked write merged with `pyarrow.ParquetWriter` bounded peak memory; `gc.collect()` was removed entirely, since Python’s own refcounting already frees the memory.
- **Q3’s `batched_top_k` results-list accumulation raised a `MemoryError` at MIND scale**. Reducing `QUERY_CHUNK_SIZE` / `BATCH_SIZE` fixed this with no functional change.
- **`cosine_similarity_subset` rebuilt a 125K-entry dictionary and renormalized the corpus subset on every one of over 12.5 million calls (Q4/Q5)**. Precomputing `id.to_idx` / `corpus_unit` once per dataset turned an unfinished run (under 1/8 of the passes done in 2h21m) into all four Q5 passes (~ 12.9 M impressions) in ~ 99 min.
- **A dict per impression cost several GB of overhead at Q4’s 12.5-million-impression scale**. Preallocated numpy arrays per scalar column avoided this entirely.
- **Both large datasets loaded together left under 0.3GB free RAM**: dead `train` rows stayed resident, and `top10_ids` grew linearly with rows scored. Loading one dataset per kernel (`EVAL_DATASETS`), dropping `train` rows after their stats are computed, and canonicalizing IDs through a shared string pool freed about 42% of `behaviors` and made `mind_large` crash-free.
- **`WinError 10055` kernel crashes could lose a scoring pass or corrupt a checkpoint mid-write**. Checkpointing every 200,000 rows, writing atomically via `os.replace`, retrying automatically, and dropping a redundant `.astype(np.float32)` copy meant a crash cost minutes rather than hours, and checkpoints were always valid.
- **Bootstrap CI over 28.5 million rows still crashed inside Jupyter even fully checkpointed**: filtering the whole `DataFrame` per slice also wasted more memory than needed. A standalone script, with no Jupyter involved, finished the computation from the checkpoints, filtering one metric column per slice at a time. `ebnerd_large`’s Q4 finished this way, with no crash.

3 Discussion

3.1 Results

Table 1 collects every core metric, recall@200, catalog coverage, AUC, and nDCG@10, for both methods across every dataset track and split. Coverage is computed once per dataset (not per split), so its value repeats across a dataset’s two rows.

Dataset	Split	Recall@200		Coverage		AUC		nDCG@10	
		BM25	Emb	BM25	Emb	BM25	Emb	BM25	Emb
EB-NeRD demo	val	3.51%	2.85%	92.0%	56.0%	0.504	0.569	0.445	0.483
EB-NeRD demo	test	2.85%	2.61%	92.0%	56.0%	0.500	0.553	0.433	0.465
EB-NeRD small	val	2.10%	1.48%	97.0%	71.5%	0.508	0.557	0.452	0.480
EB-NeRD small	test	1.82%	1.49%	97.0%	71.5%	0.501	0.550	0.434	0.464
EB-NeRD large	val	0.53%	0.29%	98.7%	77.8%	0.509	0.559	0.454	0.483
EB-NeRD large	test	0.63%	0.21%	98.7%	77.8%	0.501	0.552	0.435	0.465
MIND small	val	3.40%	2.39%	99.5%	96.7%	0.559	0.610	0.324	0.349
MIND small	test	1.61%	1.92%	99.5%	96.7%	0.556	0.595	0.336	0.361
MIND large	val	2.97%	1.99%	99.9%	99.1%	0.559	0.610	0.325	0.352
MIND large	test	1.32%	1.52%	99.9%	99.1%	0.557	0.594	0.338	0.361

Table 1: Recall@200, catalog coverage, AUC, and nDCG@10 for lexical (BM25) vs. semantic (embedding) candidate generation, across every dataset track and split. The *large* rows are the real Codabench-track catalogs (125,541/104,151 articles); recall@200 is naturally lower there than on the smaller catalogs. AUC/nDCG@10 are Q4’s offline metrics on the local *test* split, distinct from Q5’s real Codabench scores (Section 2.6).

3.2 Lexical vs. Semantic Retrieval

All numbers here were produced on one CPU-only machine (Intel Core i5-13500, 14 cores/20 threads, 16GB RAM); the pipeline’s only GPU use is Q3’s one-time Kaggle embedding computation (Section 2). BM25 leads recall@200 (defined in Section 2’s Q2 subsection as the clicked-article fraction found in the top-200 retrieved list) almost everywhere in Table 1, except MIND’s test split, where embeddings overtake it at both recall@50 (0.65% vs. 0.45%) and recall@200. BM25 also covers more of each catalog: coverage ranges 92.0–99.9% for BM25 versus 56.0–99.1% for embeddings, since mean-pooled vectors concentrate retrieval on a narrower slice.

This reverses under Q4’s AUC, scored only on the small inview set shown to a user. At both real Codabench scales, MIND large’s test split (376,471 impressions) reaches AUC 0.594 (embedding) vs. 0.557 (BM25); EB-NeRD large’s test split (12.57M impressions) reaches 0.552 vs. 0.501, the same direction. So embeddings are worse at finding the clicked article among the whole catalog, but better at ranking it once narrowed to what the user saw: recall@K and AUC answer different questions, and they agree in direction on both real submission tracks.

3.3 Dataset Differences

- **Language:** Danish vs. English. One multilingual embedding model and one Unicode-aware tokenizer handle both, with no dataset-specific code.
- **Cold-start:** EB-NeRD demo/small have zero cold-start users in val/test; MIND has 1,770 of 65,173.
- **Popularity skew:** MIND’s top 20% ever-clicked articles account for 90.2% of train clicks (EB-NeRD demo 49.2%, small 69.0%); 90–92% of each catalog is never clicked.
- **Text length:** MIND’s tokenized title+abstract (47.2 tokens avg.) is near double EB-NeRD’s (25.1–25.8), part of why MIND’s BM25 recall runs lower.

3.4 Scaling to the Full Datasets: A Real Incident

Q4’s evaluation harness hit a real scale ceiling once run against `ebnerd_large` and `mind_large` together. Loading both datasets’ full feature stores, BM25 indexes, and embedding matrices into one process left under 0.3GB free on this 15.7GB machine, and kernel crashes followed (`WinError 10055`). The crash persisted even after direct memory fixes, at a consistent point regardless of contention, evidence of a probabilistic, time-based kernel hazard rather than pure memory pressure alone. Section 2.7 lists the specific fixes this produced, alongside every other measured optimization made across the project.

The same ceiling would apply to any 10x scale-up: Q1-Q3’s per-user loops would need vectorization, the similarity matrix a real ANN index (FAISS), and single-machine parquet I/O a real feature store; BM25’s postings-list index would likely survive unchanged.